

Pi Server 실기기 E2E 테스트 지침서

- 기준일: 2026-08-12
- 대상: Raspberry Pi C++ pi-server, Hanwha Vision WiseAI, Mosquitto, Camera Snapshot API, SQLite, Qt
- 기준 브랜치: 테스트 시작 시 `git branch --show-current` 결과를 기록한다.
- 적용 슬롯: name1 → EV01, name2 → EV02, name3 → EV03, name4 → EV04

이 문서는 운영 DB와 실기기 상태를 변경하는 테스트를 포함한다. 실행 전 DB를 백업하고, 카메라 비밀번호·Gemini API Key·Telegram Token을 화면이나 문서에 노출하지 않는다.

1. 검증 목표

다음 전체 흐름을 슬롯별로 검증한다.

```
WiseAI Intrusion
→ Camera MQTT publish
→ Raspberry Pi Mosquitto
→ pi-server IVA parser/mapping
→ PARKING_SESSION ACTIVE 생성
→ Camera Snapshot API original/enhanced 다운로드
→ 저장된 최신 ROI로 crop
→ IMAGE_LOG 및 Gemini OCR 연결
→ Qt OCCUPIED/이미지 표시
→ WiseAI Exit
→ 20초 확인 대기
→ 조기 출차면 이미지/IMAGE_LOG 삭제
  또는 위반 세션이면 증거 보존
```

완료 판정은 로그 한 줄이 아니라 MQTT 원문, 서버 로그, 파일, SQLite, HTTP API, Qt 화면을 함께 확인한다.

2. 현재 운영 계약

구분	값/정책
Pi HTTP	http://<PI_IP>:8080
MQTT Broker	Raspberry Pi Mosquitto, TCP 1883
Camera MQTT 구독	+/onvif-ej/#
점유 입력	PARKING_OCCUPANCY_SOURCE=CAMERA_IVA
입차 권한	WiseAI 또는 호환 Publication의 INTRUSION
출차 권한	카메라 원본 WiseAI Data.Action=Exit 만 인정
출차 확인	기본 20초, CAMERA_IVA_EXIT_CONFIRM_MS
촬영	Camera Snapshot API 우선
RTSP fallback	운영 설정에 따라 결정; 현재 API 전용 모드에서는 보통 false
이미지	슬롯/촬영 사유별 original/, enhanced/
조기 출차	violation_at IS NULL: 예약/OCR/이미지/IMAGE_LOG 정리
위반 출차	violation_at IS NOT NULL: 위반 증거 보존

중요한 차이:

카메라 WiseAI 영역(name1~name4)

→ 감지와 MQTT 이벤트 발생 영역

Qt/Pi ROI 설정

→ Snapshot API 이미지에서 저장·OCR할 crop 영역

Qt ROI가 없어도 WiseAI 감지는 발생할 수 있지만, 유효한 crop과 촬영은 실패한다. 반대로 Qt ROI를 저장해도 카메라 WiseAI 이벤트가 자동 활성화되지는 않는다.

3. 테스트 전 안전 점검

3.1 코드와 프로세스 상태 기록

```
git branch --show-current && git status --short && git log -5 --oneline
```

```
pgrep -a pi-server || true
```

테스트 시작 전에 예상하지 못한 `pi-server` 가 있으면 중복 실행하지 않는다. 종료는 다음 한 줄을 사용한다.

```
pkill -INT -x pi-server 2>/dev/null || true
```

3.2 DB 백업

```
./tools/backup_db.sh
```

백업 스크립트가 현재 환경과 맞지 않으면 서버를 중지한 상태에서 `data/db/parking.db` 를 별도 안전 경로에 복사한다.

3.3 공개/비밀 설정 확인

값의 존재 여부만 확인하고 비밀번호는 출력하지 않는다.

```
grep -E '^((CAMERA_EVENT_SUB_TOPIC|PARKING_OCCUPANCY_SOURCE|CAMERA_SNAPSHOT_API_ENABLED|CAMERA_SNAPSHOT_API_RTSP_FALLBACK|CAMERA_IVA_EXIT_CONFIRM_MS|IVA_EV0[1-4]_AREA_NAME|BESTSHOT_ENABLED))=' .env.public
```

```
for key in CAMERA_OPEN_API_BASE CAMERA_IMAGE_BASE CAMERA_API_USERNAME CAMERA_API_PASSWORD;
do grep -q "^${key}=" .env.private && echo "$key=SET" || echo "$key=MISSING"; done
```

기대값:

```
CAMERA_EVENT_SUB_TOPIC=/onvif-ej/#
PARKING_OCCUPANCY_SOURCE=CAMERA_IVA
CAMERA_SNAPSHOT_API_ENABLED=true
CAMERA_SNAPSHOT_API_RTSP_FALLBACK=false
IVA_EV01_AREA_NAME=name1
IVA_EV02_AREA_NAME=name2
IVA_EV03_AREA_NAME=name3
IVA_EV04_AREA_NAME=name4
BESTSHOT_ENABLED=false
```

3.4 네트워크와 서비스

```
hostname -I && systemctl is-active mosquitto && ss -lntp | grep ':1883'
```

카메라 MQTT Client의 Broker Address는 위에서 확인한 Pi IP여야 한다. `Connected` 표시는 Broker 접속만 보장하며 IVA Event Rule 발행까지 보장하지 않는다.

4. 카메라 설정 점검

4.1 WiseAI 영역

CH1에서 다음 매핑을 확인한다.

WiseAI 영역	서버 슬롯	Video token
name1	EV01	vs-0
name2	EV02	vs-0
name3	EV03	vs-0
name4	EV04	vs-0

각 영역에서 Intrusion 과 운영에 필요한 Exit 분석이 활성화되고 Apply/Save 되었는지 확인한다.

4.2 MQTT Client

Address: <PI_IP>
Port: 1883
Transport: TCP
Protocol: MQTT
Username/Password: Broker 정책에 따름
Enable: On
Status: Connected

4.3 커스텀 Publication을 사용하는 경우

EV02 Intrusion 예시:

Name: IVA_EV02_INTRUSION
Enable: On
Default topic prefix: Off
Topic: cam01/onvif-ej/iva/vs-0/EV02/intrusion
QoS: 1
Retain: Off

```
{"schema": "smart-parking-iva-v1", "camera_id": "cam01", "video_source_token": "vs-0", "rule_name": "name2", "slot_id": "EV02", "event_type": "IVA_AREA", "action": "INTRUSION", "active": true}
```

Publication 생성 후 Event Rule에서 해당 Publication을 MQTT Action으로 선택하고 Rule Enable 및 Apply/Save를 확인한다.

커스텀 고정 EXIT Publication은 서버가 권위 있는 출처로 인정하지 않는다. 잘못 연결된 고정 EXIT가 입차와 동시에 발행되어 사진을 즉시 삭제하던 문제를 막기 위한 정책이다. 출처 검증은 원본 WiseAI Data.Action=Exit 로 수행한다.

5. 1단계: 서버 없이 카메라 MQTT 원문 검증

Pi 서버를 끈 상태에서 카메라→Broker만 분리 검증한다.

전체 IVA 감시:

```
mosquitto_sub -h localhost -p 1883 -v -t '+/onvif-ej/OpenApp/WiseAI/IvaArea/#' -t 'cam01/onvif-ej/iva/#'
```

EV02 Intrusion 성공 예:

```
cam01/onvif-ej/iva/vs-0/EV02/intrusion { ... "rule_name":"name2", ... "action":"INTRUSION", "active":true }
```

원본 WiseAI 성공 예:

```
.../onvif-ej/OpenApp/WiseAI/IvaArea/&vs-0/name2  
Data.Action=Intrusion 또는 Data.Action=Exit
```

테스트할 때 영역을 완전히 비워 정상 상태로 되돌린 뒤 다시 진입한다. 이미 Intrusion 상태라면 같은 상태를 유지하는 동안 새 이벤트가 발생하지 않을 수 있다.

판정:

증상	원인 범위
일반 ONVIF 이벤트도 없음	카메라 MQTT Client/Broker/network
영역은 빨간색이나 IVA MQTT 없음	카메라 Event Rule/MQTT Action/Apply/Enable
커스텀 토픽만 수신	Publication은 정상, 원본 WiseAI Exit 별도 확인 필요
원본 IvaArea/nameN 수신	카메라 네이티브 IVA 경로 정상

6. 2단계: Pi 서버 기동

```
./run_server.sh
```

다른 터미널에서 확인한다.

```
curl -fsS http://127.0.0.1:8080/api/v1/health && echo
```

```
tail -n 0 -F data/logs/pi-server.log | grep --line-buffered -E 'IVA_OCCUPANCY|IVA_EXIT|CAMERA_SNAPSHOT_API|EVIDENCE_CAPTURE|CAPTURE_SCHED|PLATE_PREPROCESS|Gemini|WARN|ERROR'
```

정상 시작 핵심 로그:

```
camera snapshot API ready
camera IVA occupancy enabled
MQTT connected
MQTT subscribed: +/onvif-ej/#
waiting for camera MQTT events
```

7. 3단계: Intrusion → OCCUPIED → 촬영

1. 테스트 슬롯에 종료되지 않은 기존 세션이 없는지 확인한다.
2. 카메라 영역을 완전히 비운다.
3. nameN 영역에 차량 또는 검출 대상을 진입시킨다.
4. MQTT 원문과 서버 로그를 함께 확인한다.

정상 로그 예:

```
[IVA_OCCUPANCY] slot=EV02 state=OCCUPIED active_areas=1/1 known_areas=1 object=<object_id>
```

동일 Intrusion 반복은 다음처럼 중복 세션 없이 무시될 수 있으며 정상이다.

```
[IVA_OCCUPANCY] duplicate ignored slot=EV02 object=<object_id>
```

7.1 세션 DB 확인

```
sqlite3 -header -column data/db/parking.db "SELECT session_id,slot_id,status,entry_time,violation_at,exit_time FROM PARKING_SESSION WHERE slot_id='EV02' ORDER BY session_id DESC LIMIT 5;"
```

입차 직후 기대값:

```
slot_id=EV02
status=ACTIVE
violation_at=NULL
exit_time=NULL
```

7.2 이미지 파일 확인

```
find data/snapshots/ch1/EV02 -type f -printf '%TY-%Tm-%Td %TH:%TM:%TS %p\n' | sort | tail -20
```

기본 저장 구조:

```
data/snapshots/ch1/EV02/  
├─ occupancy_start/original/  
├─ occupancy_start/enhanced/  
├─ hall_30s/original/  
├─ hall_30s/enhanced/  
├─ hall_60s/original/  
├─ hall_60s/enhanced/  
└─ overstay/{original,enhanced}/
```

7.3 IMAGE_LOG 확인

위 조회에서 얻은 실제 세션 ID로 <SESSION_ID> 를 바꾼다.

```
sqlite3 -header -column data/db/parking.db "SELECT image_id,session_id,evidence_reason,captured_at,original_image_path,enhanced_image_path FROM IMAGE_LOG WHERE session_id=<SESSION_ID> ORDER BY captured_at,image_id;"
```

파일 경로가 비어 있거나 존재하지 않는 파일을 가리키면 실패다.

7.4 ROI crop 확인

```
curl -fsS http://127.0.0.1:8080/api/v1/settings/parking-slots/EV02/roi && echo
```

저장된 original/enhanced를 열어 Qt에서 설정한 EV02 영역과 일치하는지 확인한다. 해상도 2592x1520 을 하드 코딩하지 않고 실제 이미지 크기에 정규화 ROI를 적용해야 한다.

8. 4단계: HTTP API와 Qt 확인

```
curl -fsS http://127.0.0.1:8080/api/v1/parking-sessions/<SESSION_ID>/images && echo
```

확인 사항:

- items 가 captured_at 오름차순인가.
- evidence_reason 이 정확한가.
- 절대 파일 경로가 응답에 노출되지 않는가.
- original_url 이 HTTP 200과 JPEG를 반환하는가.
- enhanced 파일이 있으면 enhanced_url 이 HTTP 200을 반환하는가.

- Qt에서 EV02가 OCCUPIED로 표시되는가.
- Qt 이미지 화면이 동일 세션의 이미지를 표시하는가.

MQTT 상태 확인:

```
mosquitto_sub -h localhost -p 1883 -v -t 'parking/v1/events/+' -t 'parking/v1/state/+'
```

9. 5단계: 조기 출차 삭제 검증

이 테스트는 `violation_at IS NULL` 인 새 세션에서 수행한다.

1. Intrusion으로 ACTIVE 세션과 이미지를 만든다.
2. 제한시간 전에 카메라 원본 WiseAI Exit를 발생시킨다.
3. `CAMERA_IVA_EXIT_CONFIRM_MS` 동안 기다린다.
4. 확인 중 같은 슬롯 Intrusion을 다시 발생시키지 않는다.

정상 로그:

```
[IVA_OCCUPANCY] slot=EV02 state=VACANT_CANDIDATE ...
[IVA_EXIT] pending slot=EV02 ... confirm_ms=20000
[IVA_EXIT] confirmed slot=EV02
```

DB 확인:

```
sqlite3 -header -column data/db/parking.db "SELECT session_id,slot_id,status,entry_time,violation_at,exit_time FROM PARKING_SESSION WHERE session_id=<SESSION_ID>; SELECT image_id,session_id,evidence_reason,original_image_path FROM IMAGE_LOG WHERE session_id=<SESSION_ID>;"
```

통과 기준:

- `exit_time` 이 기록된다.
- `status=ENDED` 다.
- `violation_at` 은 NULL이다.
- 해당 세션의 `IMAGE_LOG` 가 제거된다.
- 해당 세션의 원본/개선 파일이 제거된다.
- 예약된 30/60초 촬영 및 OCR 결과가 뒤늦게 세션을 되살리지 않는다.

10. 6단계: Exit flap 취소 검증

1. ACTIVE 세션에서 원본 WiseAI Exit를 발생시킨다.
2. 20초 확인시간 안에 같은 슬롯 Intrusion을 다시 발생시킨다.

정상 로그:


```
[IVA_EXIT] pending slot=EV02 ...  
[IVA_EXIT] canceled by INTRUSION slot=EV02 ...  
[IVA_OCCUPANCY] slot=EV02 state=OCCUPIED ...
```

통과 기준: 기존 `session_id`가 유지되고 `exit_time`이 생기지 않으며 파일이 삭제되지 않는다.

11. 7단계: 장기점유 및 증거 보존 검증

운영 1시간을 기다리지 않도록 REST API로 최소 허용값 60초를 사용한다. 먼저 기존 값을 기록한다.

```
curl -fsS http://127.0.0.1:8080/api/v1/settings/overstay-threshold && echo
```

```
curl -fsS -X PUT http://127.0.0.1:8080/api/v1/settings/overstay-threshold -H 'Content-Type: application/json' -d '{"thresholdSeconds":60}' && echo
```

새 세션을 시작하고 60초 이상 유지한다. 기대 결과:

- `PARKING_SESSION.status=VIOLATION`
- `violation_at` 기록
- `OVERSTAY_EVIDENCE IMAGE_LOG` 1건
- `parking/v1/events/{slot}`에 장기점유 이벤트
- Qt 경고 표시

```
sqlite3 -header -column data/db/parking.db "SELECT session_id,slot_id,status,violation_at,exit_time FROM PARKING_SESSION WHERE session_id=<SESSION_ID>; SELECT image_id,evidence_reason,captured_at,original_image_path,enhanced_image_path FROM IMAGE_LOG WHERE session_id=<SESSION_ID> ORDER BY captured_at,image_id;"
```

이후 원본 WiseAI Exit를 발생시키고 확인시간을 기다린다. 위반 세션은 출차 후 `exit_time`이 기록되더라도 `violation_at`, `EVENT_LOG`, `IMAGE_LOG` 및 증거 파일이 보존되어야 한다.

테스트 종료 후 처음 기록한 기준시간으로 반드시 복원한다. 운영 기본값 예:

```
curl -fsS -X PUT http://127.0.0.1:8080/api/v1/settings/overstay-threshold -H 'Content-Type: application/json' -d '{"thresholdSeconds":3600}' && echo
```

12. 4개 슬롯 반복표

슬롯	WiseAI Rule	Intrusion MQTT	ACTIVE 세션	ROI crop	Exit confirmed	조기 삭제	위반 보존	결과
EV01	name1	☐	☐	☐	☐	☐	☐	PASS / FAIL
EV02	name2	☐	☐	☐	☐	☐	☐	PASS / FAIL
EV03	name3	☐	☐	☐	☐	☐	☐	PASS / FAIL
EV04	name4	☐	☐	☐	☐	☐	☐	PASS / FAIL

13. 장애 판정표

관찰 결과	먼저 확인할 위치
영역이 빨강게 되지 않음	WiseAI 영역/검출 조건/적용 상태
영역은 빨강지만 Broker에 IVA 없음	Event Rule, MQTT Action, Publication Enable, Apply/Save
Broker에는 IVA가 있지만 서버 로그 없음	pi-server 실행, 구독 토픽, parser/mapping rejection 로그
camera/token/rule mapping not found	parking_slots.json의 cam01 + vs-0 + nameN
OCCUPIED지만 사진 없음	Snapshot API 준비 로그, ROI 존재, API 인증/주소
ROI mapping not found	Qt ROI PUT 또는 슬롯별 ROI 설정
촬영 직후 사진 삭제	원본/고정 EXIT 동시 발행, 조기 출차 처리 로그
오래된 사진 수신	Snapshot API run_id, generate 직후 GET, 카메라 API 상태
IMAGE_LOG만 있고 파일 없음	파일/DB 일관성 실패; 테스트 실패
반복 Intrusion으로 세션 중복	중복 억제/활성 세션 조회 실패; 테스트 실패
Exit 후 사진이 남음	violation_at 존재 여부부터 확인

14. 자동 테스트와 빌드 회귀 확인

실기기 시험 전후 코드가 변경됐다면 다음을 실행한다.

```
cmake -S . -B cmake-build -DCMAKE_BUILD_TYPE=Release && cmake --build cmake-build -j2
```

```
ctest --test-dir cmake-build --output-on-failure
```

IVA 관련 빠른 회귀:

```
ctest --test-dir cmake-build -R 'camera-iva-event-test|camera-iva-occupancy-integration-test|camera-snapshot-api-client-test|hall-capture-pipeline-test|http-api-test' --output-on-failure
```

15. 제출용 증거 목록

다음 화면을 같은 세션 ID와 시각이 보이도록 캡처한다.

1. 카메라 nameN Intrusion 감지 화면
2. Mosquitto의 원본 또는 커스텀 Intrusion 수신
3. [IVA_OCCUPANCY] ... OCCUPIED 로그
4. original/enhanced ROI crop 파일
5. PARKING_SESSION ACTIVE 행
6. IMAGE_LOG 행
7. HTTP 세션 이미지 JSON과 실제 이미지 HTTP 200
8. Qt OCCUPIED 및 이미지 표시
9. 조기 Exit pending/confirmed 로그와 이미지 삭제 결과
10. 장기점유 VIOLATION, violation_at, OVERSTAY_EVIDENCE 및 Qt 경고
11. 전체 CTest 통과 화면

16. 테스트 종료

```
pkill -INT -x pi-server 2>/dev/null || true
```

확인:

```
pgrep -a pi-server || echo 'pi-server stopped'
```

장기점유 기준시간, 카메라 Event Rule, 임시 Publication, 테스트용 활성 세션을 원래 운영 상태로 복구한다. DB 행이나 사진을 직접 삭제하기 전에 백업과 세션 정책을 확인한다.

17. 결과 기록 양식

테스트 일시:
테스터:
Git branch / HEAD:
Pi IP:
Camera IP:
Camera model / firmware:
대상 슬롯:
PARKING_SESSION.session_id:
MQTT Intrusion: PASS / FAIL
OCCUPIED 전이: PASS / FAIL
Snapshot original/enhanced: PASS / FAIL
ROI crop: PASS / FAIL
IMAGE_LOG: PASS / FAIL
Gemini OCR: PASS / FAIL / SKIP
Qt 상태/이미지: PASS / FAIL / SKIP
Exit 확인: PASS / FAIL
조기 출차 삭제: PASS / FAIL / N/A
위반 증거 보존: PASS / FAIL / N/A
CTest: PASS / FAIL
장애 및 로그:
최종 판정: PASS / CONDITIONAL PASS / FAIL